



ROBOTINO PROJECT

Suraj Karki

19.09.2025

Prof. Dr.-Ing. Christian Schwede; M.Sc. Philip Maurice Scheidig; B.Sc Roman Sliwinski



INTRODUCTION

- Robotino AGV → mobile robot for education & research
- Equipped with sensors (**SICK LMS100**, **optical encoders**, **infrared sensor and camera**) for autonomous navigation.
- It has omnidirectional movement for flexible navigation.





PROBLEM STATEMENT

- Currently, generation of path network for Robotino is **manual and slow**
- It relies heavily on **expert knowledge**
- Changing paths for new factory layouts takes a lot of **time** and **effort**.





OBJECTIVE

- To generate a map of the environment including the stations of the IoT factory
- To generate an optimized path network based on recordings





METHODS & IMPLEMENTATIONS

■ SLAM (Simultaneous Localization and Mapping)

- It is the process by which a mobile robot build a map of the environment and, at the same time use this map to compute its location simultaneously. ¹



1. Bailey, Tim, and Hugh Durrant-Whyte. "Simultaneous localization and mapping (SLAM): Part II." IEEE robotics & automation magazine 13.3 (2006): 108-117.



METHODS & IMPLEMENTATIONS

■ Types of SLAM:

■ Filtering

- State is estimated “on the go” with latest measurements
- For example, Kalman Filter, EKF, Particle Filtering, etc.

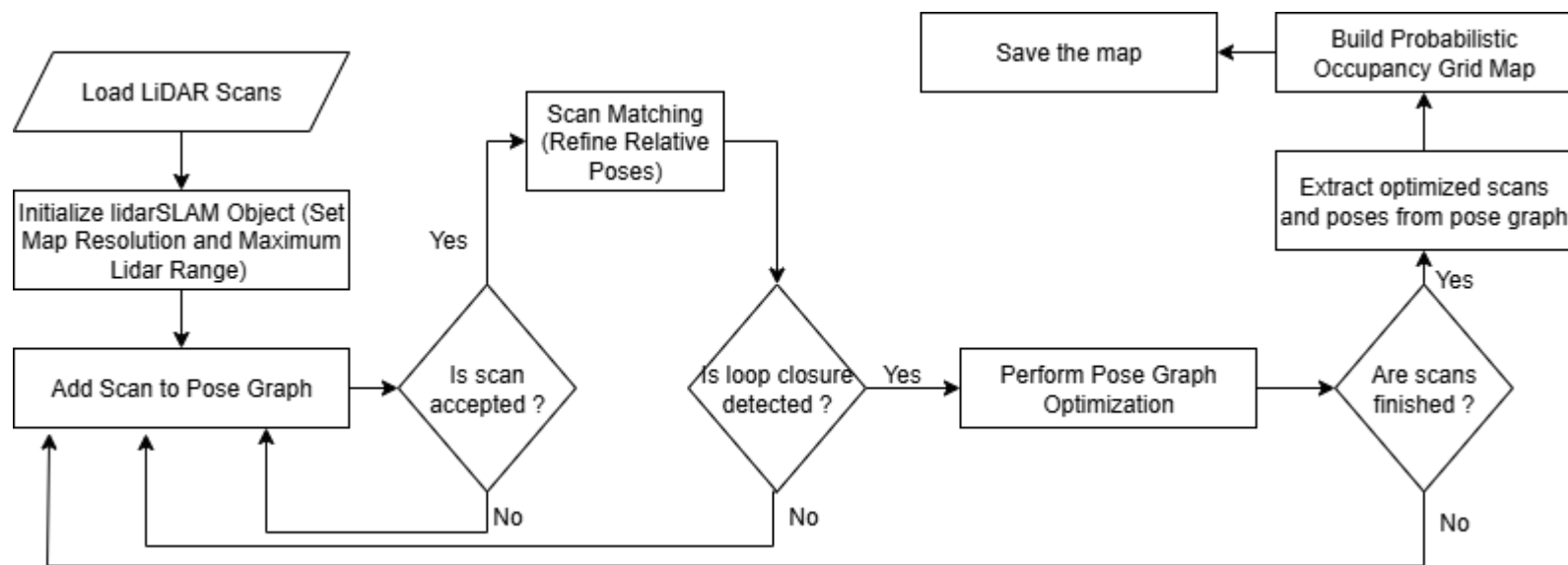
■ Smoothing

- Full trajectories are estimated using complete set of measurements
- For example, Graph-based SLAM with Pose Graph Optimization, etc.



METHODS & IMPLEMENTATIONS

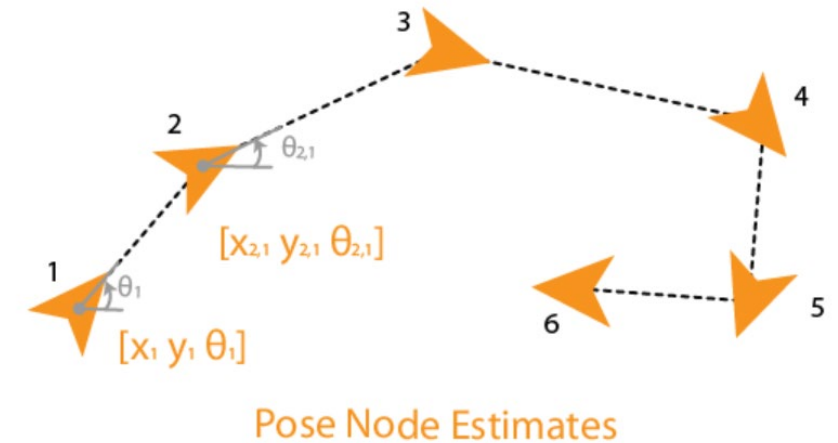
Workflow for Map Generation (Graph-based SLAM)



METHODS & IMPLEMENTATIONS

■ Pose Graph

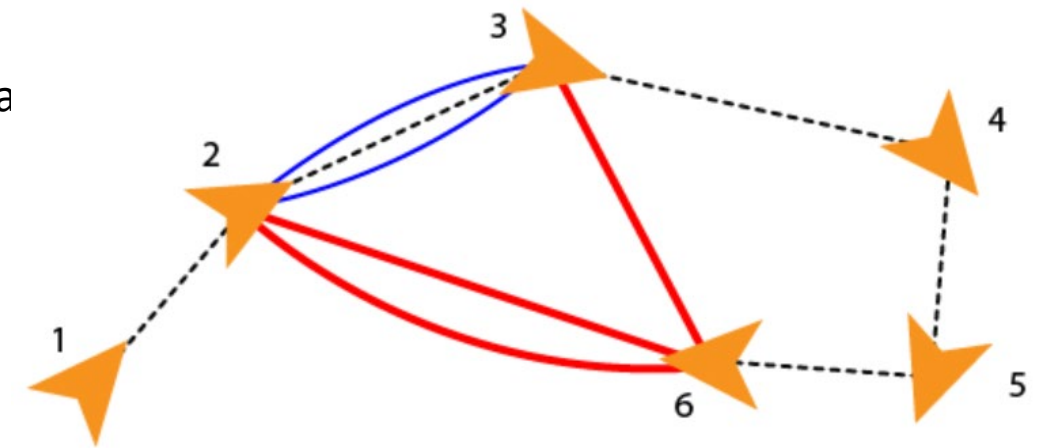
- A pose graph contains nodes connected by edges.
- Nodes → robot poses (position + orientation) at different times
- Edges → constraints between poses (odometry constraint, loop closures)
- These edge constraints also represent the uncertainty associated with that measurement.



METHODS & IMPLEMENTATIONS

■ Loop Closure

- It tells the SLAM system that the robot has returned to a previously visited area.
- It helps to correct errors (drift in trajectory) in the map.
- Connecting two nodes that are not next to each other (non-sequential), creates a loop closure.



Loop closures and multiedges



METHODS & IMPLEMENTATIONS

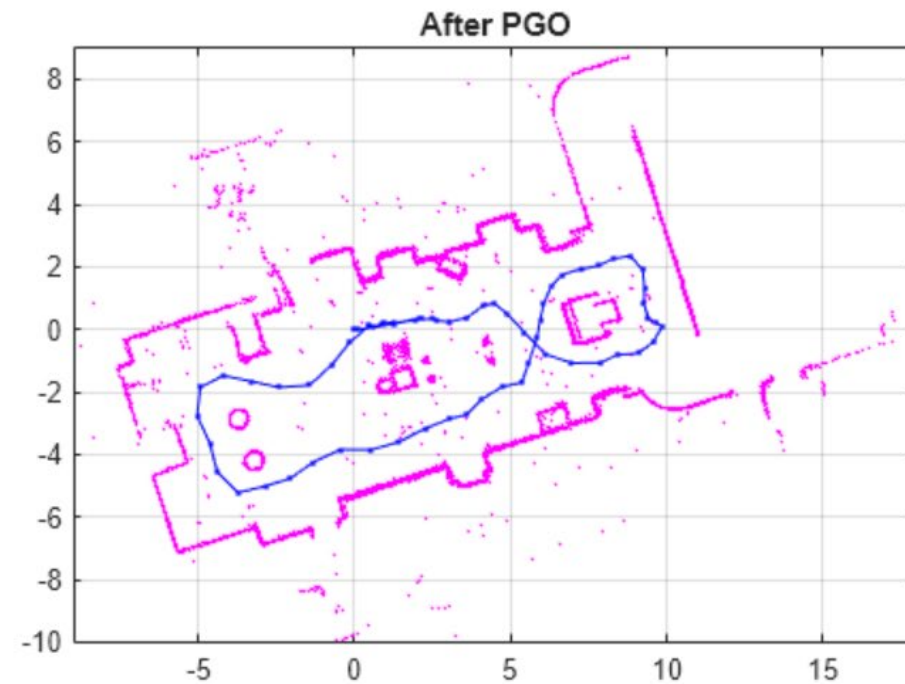
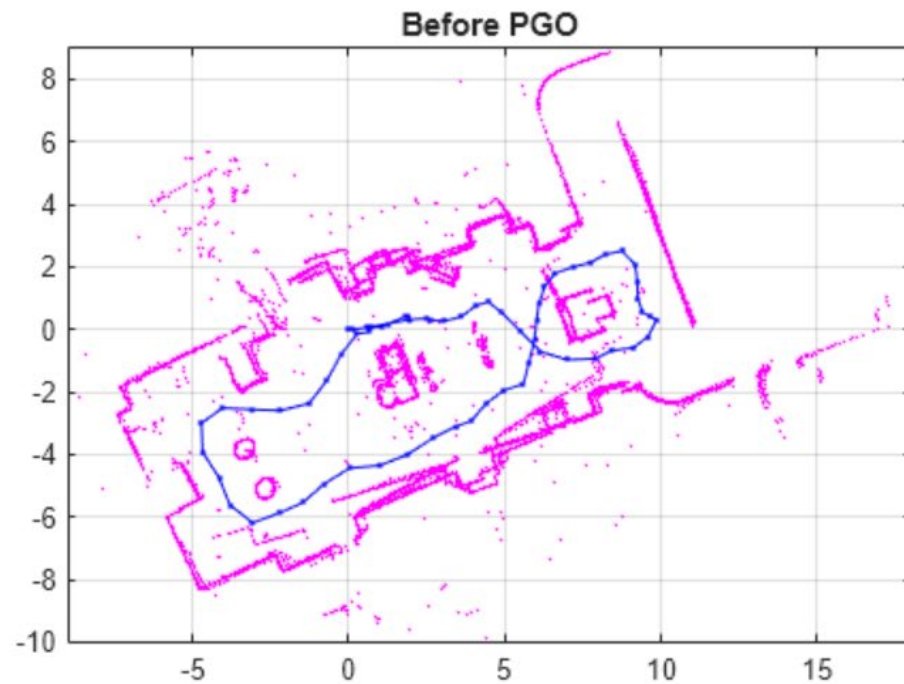
■ Pose Graph Optimization

- It is a technique to improve the estimated poses (positions and orientations) of a robot as it moves through an environment.
- Goal: to minimize the errors in the estimated poses by considering the relationships and constraints between them.
- This is typically achieved by minimizing a global error function.
- This function measures the difference between the observed constraints and the predicted constraints based on the current pose estimates.
- The optimization process adjusts the poses to reduce this error, resulting in a more accurate and consistent set of poses.

METHODS & IMPLEMENTATIONS

■ Pose Graph Optimization

Map of the Environment and Robot Trajectory





METHODS & IMPLEMENTATIONS

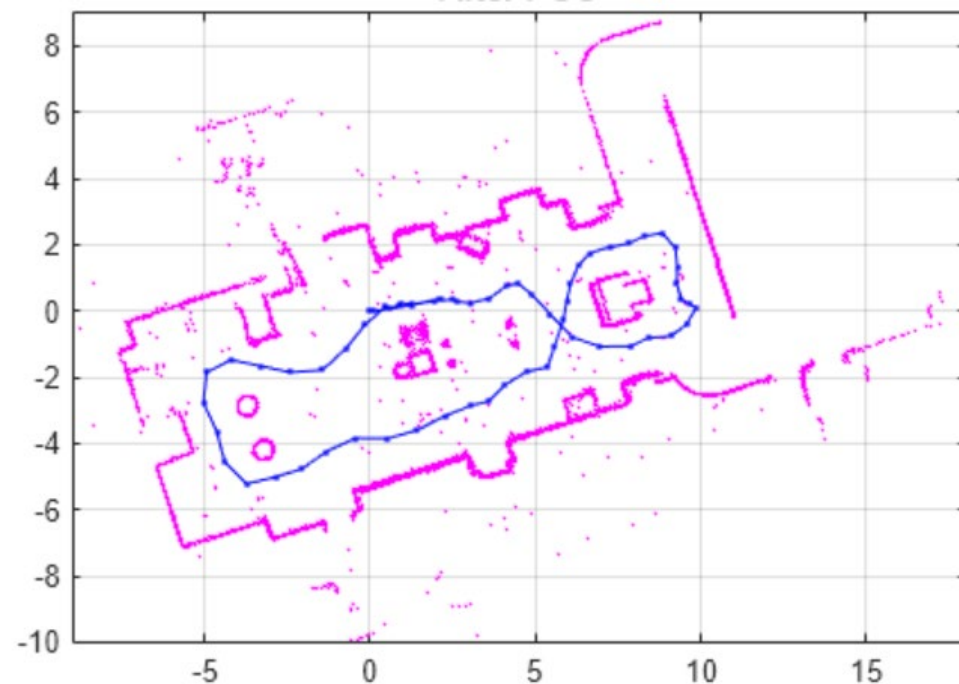
■ Occupancy Grid Map

- A way to represent the environment as a grid of cells.
- Each cell in the occupancy grid has a value representing the probability of the occupancy of that cell
- Value close to 1 → cell likely occupied (obstacle).
- Value close to 0 → cell likely free (no obstacle).
- Value = 0.5 → unknown

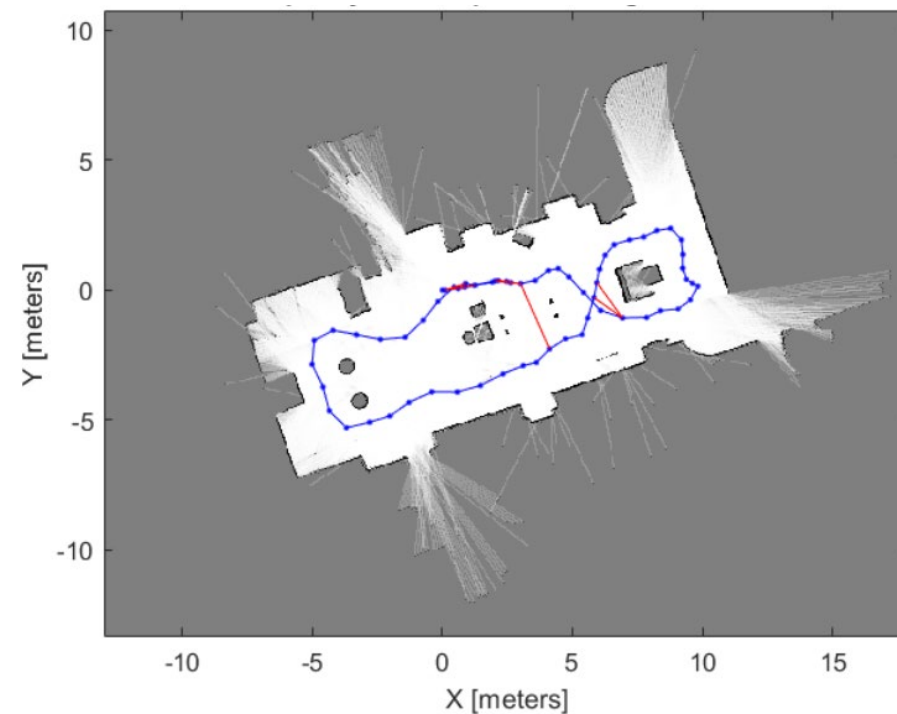


METHODS & IMPLEMENTATIONS

Occupancy Grid Map



Optimized Scans & Poses

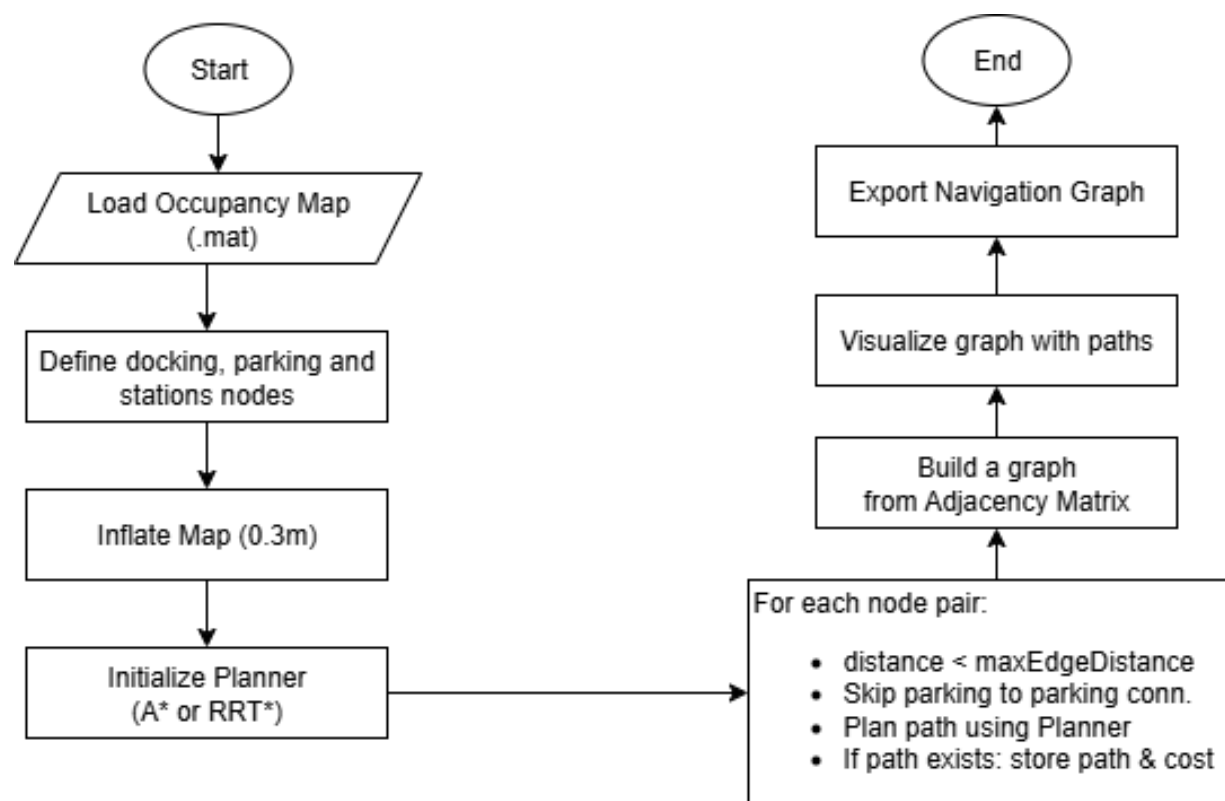


2D Probability Occupancy Grid
Map



METHODS & IMPLEMENTATIONS

Workflow for Optimized Path-Network Generation





METHODS & IMPLEMENTATIONS

■ Path Planning

- Finding a path from the current position to the goal position given a map of the environment.

■ Path Planning Algorithms

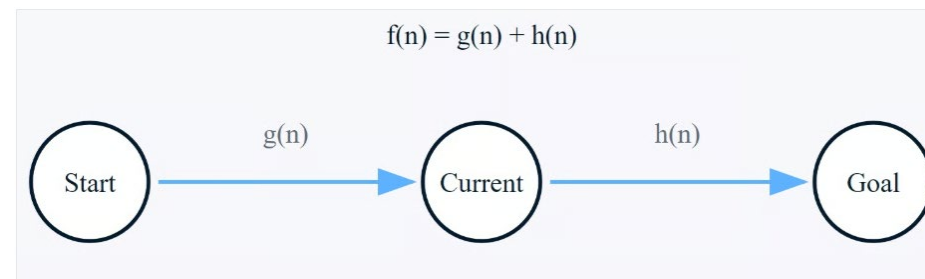
- Search-based: A*
- Sampling-based: RRT, RRT*



METHODS & IMPLEMENTATIONS

I A*

- I A popular graph traversal path planning algorithm.
- I Heuristic-Based.
- I Evaluation function, $f(n)=g(n)+h(n)$
 - I Where, $f(n)$ is the estimated total cost of the path through node n .
 - I $g(n)$ is actual cost from the start node to n .
 - I $h(n)$ is estimated cost (heuristic) from n to the goal node.





METHODS & IMPLEMENTATIONS

I A*

- I Optimality: Guarantees the shortest path if the heuristic is admissible (never overestimates).
- I Completeness: If a path exists, A* will always find it.
- I The performance of A* depends heavily on the heuristic function chosen.
- I Works with different heuristics like Manhattan distance, Euclidean distance, or custom heuristics depending on the problem.



METHODS & IMPLEMENTATIONS

■ Workings of A*

■ Initialization

■ Setting up two essential lists:

- “open” list: initialize with starting node
- “closed” list: begins empty

■ Each node stores 4 critical information:

- g: The actual cost from the start node
- h: The estimated cost to the goal
- f: The sum of g and h
- parent: Reference to the previous node (for path reconstruction)



METHODS & IMPLEMENTATIONS

■ Workings of A*

■ Main Loop:

■ The main loop will continue until either:

■ The goal is reached (success)

■ The open list becomes empty (failure - no path exists)

■ During each iteration, the algorithm:

■ Selects the most promising node (lowest f value) from the open list

■ Moves it to the closed list

■ Examines all neighboring nodes



METHODS & IMPLEMENTATIONS

■ Workings of A*

■ **Neighbor Examination:**

- For each neighbor, the algorithm:
 - Skips nodes already in the closed list
 - Calculates a tentative g score
 - Updates node values if a better path is found
 - Adds new nodes to the open list



METHODS & IMPLEMENTATIONS

■ Workings of A*

■ **Path Reconstruction:**

- Once the goal is reached, the algorithm works backward through the parent references to construct the optimal path from start to goal.



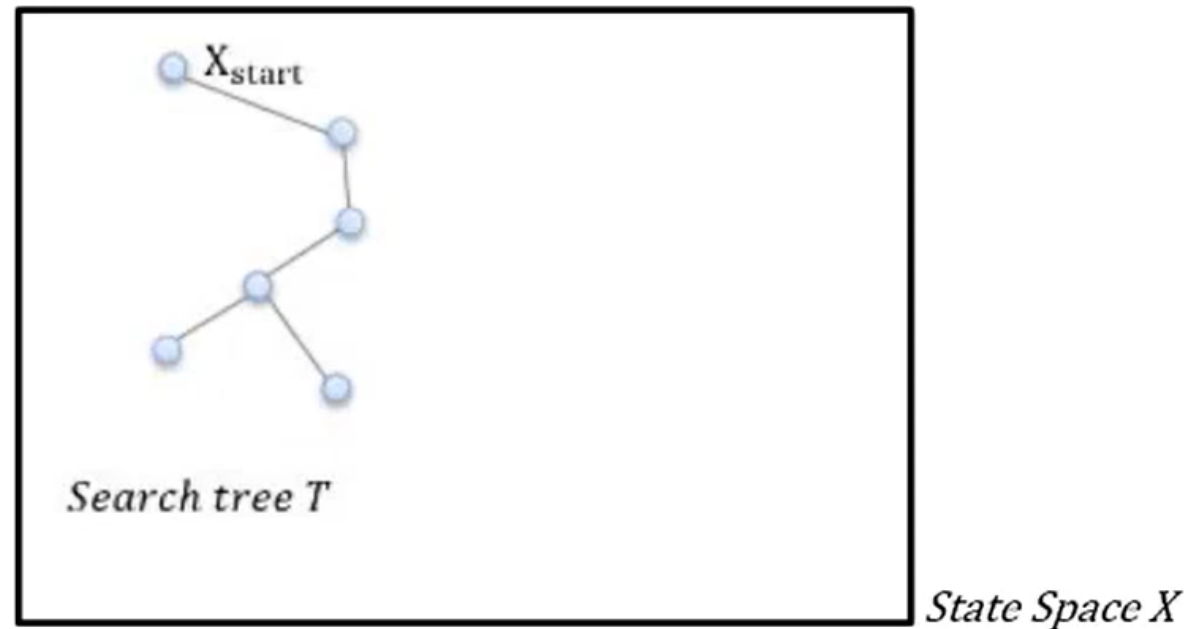
METHODS & IMPLEMENTATIONS

- RRT (Rapidly Exploring Random Tree)
 - Sampling-based path planning algorithm
 - Grows a random tree by sampling points in the space
 - and connect them to the nearest existing node.
 - Efficiently explores large, high-dimensional, or obstacle-filled environments.
 - if a path exists, RRT will find it given enough time.
 - basic RRT does not ensure the shortest path

METHODS & IMPLEMENTATIONS

Workings of RRT

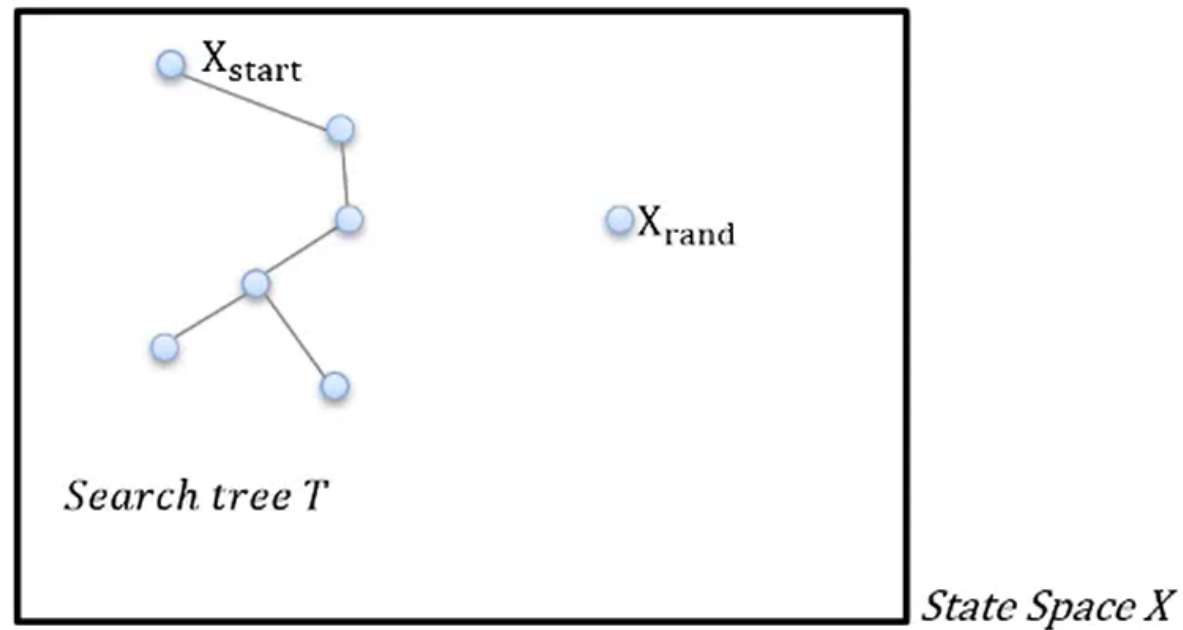
- Define the State Space:** Represent the environment (e.g., SE2 for 2D with orientation).
- Initialize the Tree** – Start with the root node at the robot's initial position.



METHODS & IMPLEMENTATIONS

Workings of RRT

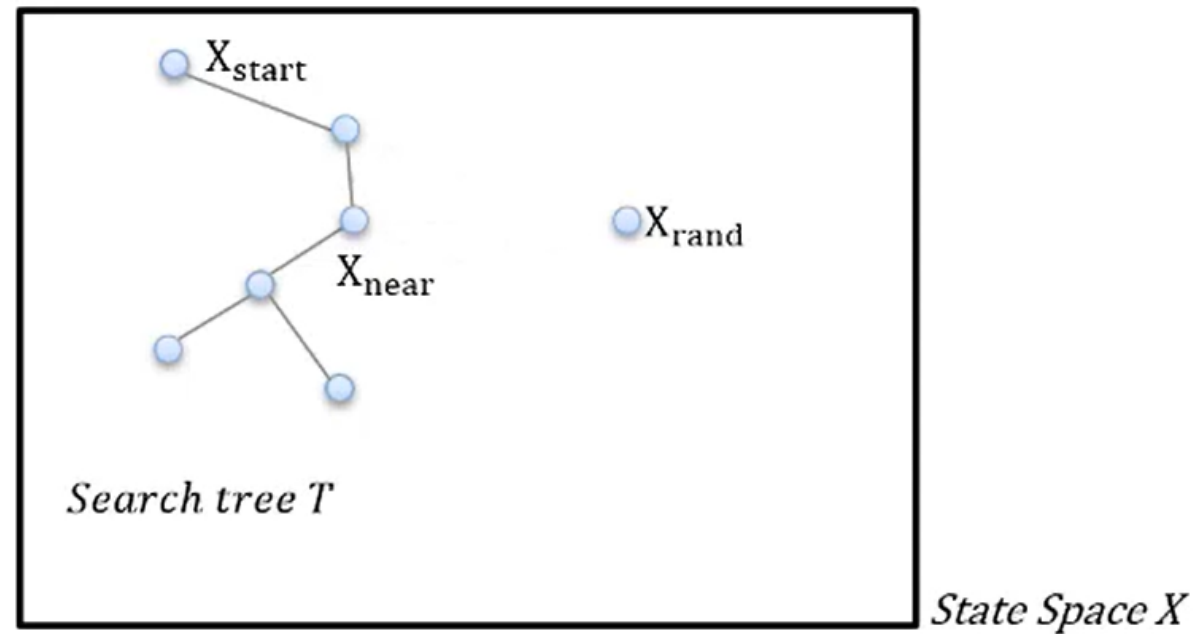
- Random Sampling** – Sample a random state X_{rand} within the state space.



METHODS & IMPLEMENTATIONS

Workings of RRT

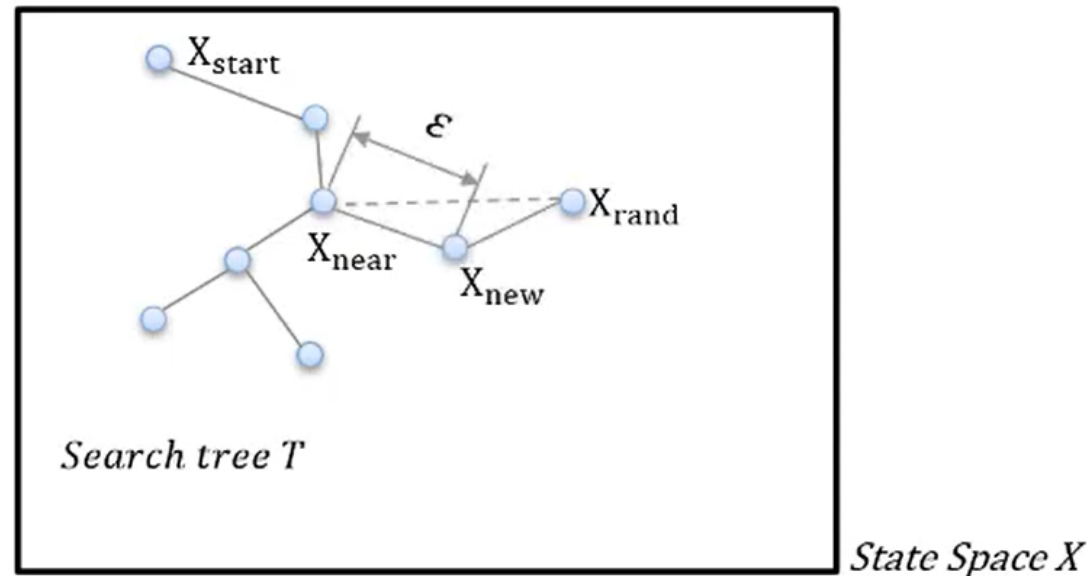
- Find Nearest Node** – Identify the tree node X_{near} closest to X_{rand} .



METHODS & IMPLEMENTATIONS

Workings of RRT

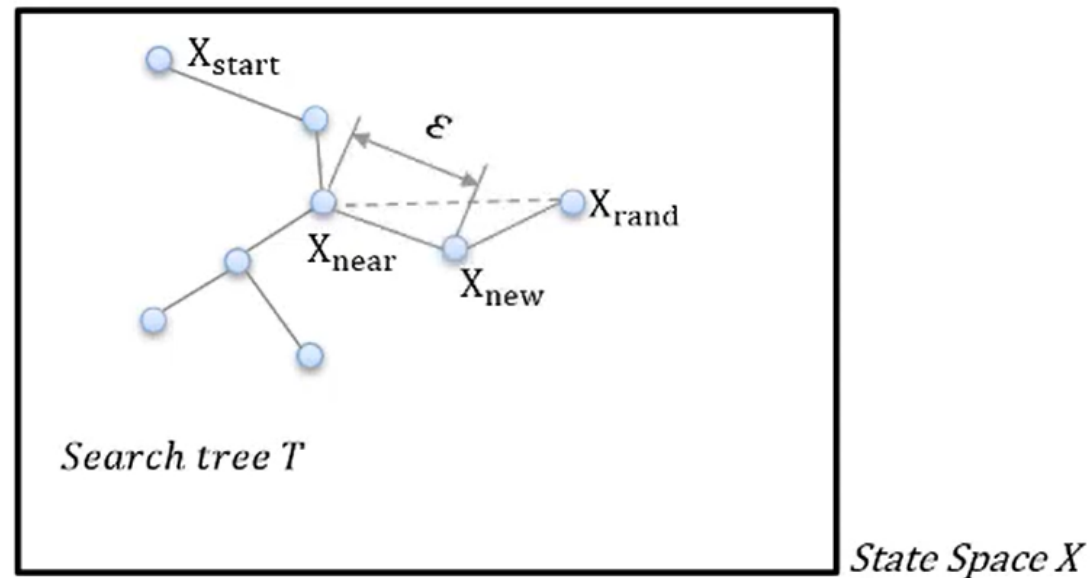
- Generate New Node** – Extend from X_{near} toward X_{rand} to create X_{new} , using feasible motion models.



METHODS & IMPLEMENTATIONS

Workings of RRT

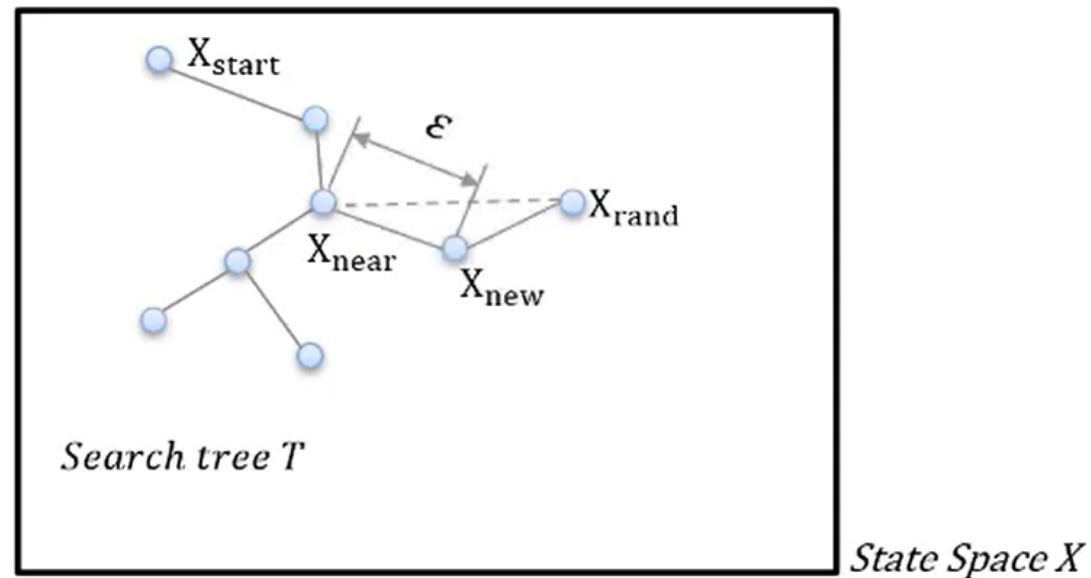
- Collision Checking** – Validate that the motion from X_{near} to X_{new} is collision-free.



METHODS & IMPLEMENTATIONS

Workings of RRT

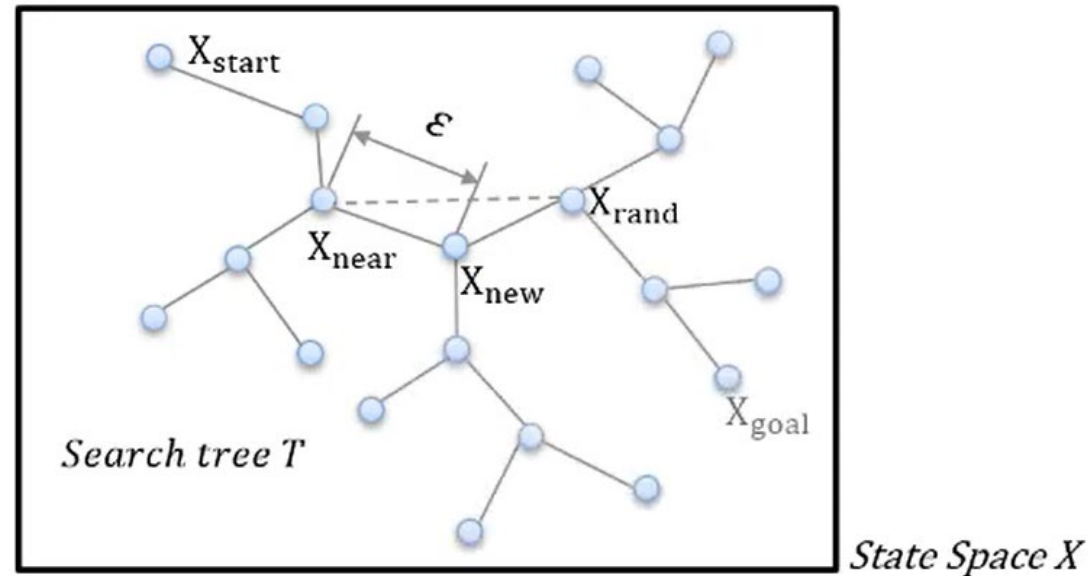
- Add Node to Tree** – If valid, add X_{new} and connect it to X_{near} .



METHODS & IMPLEMENTATIONS

Workings of RRT

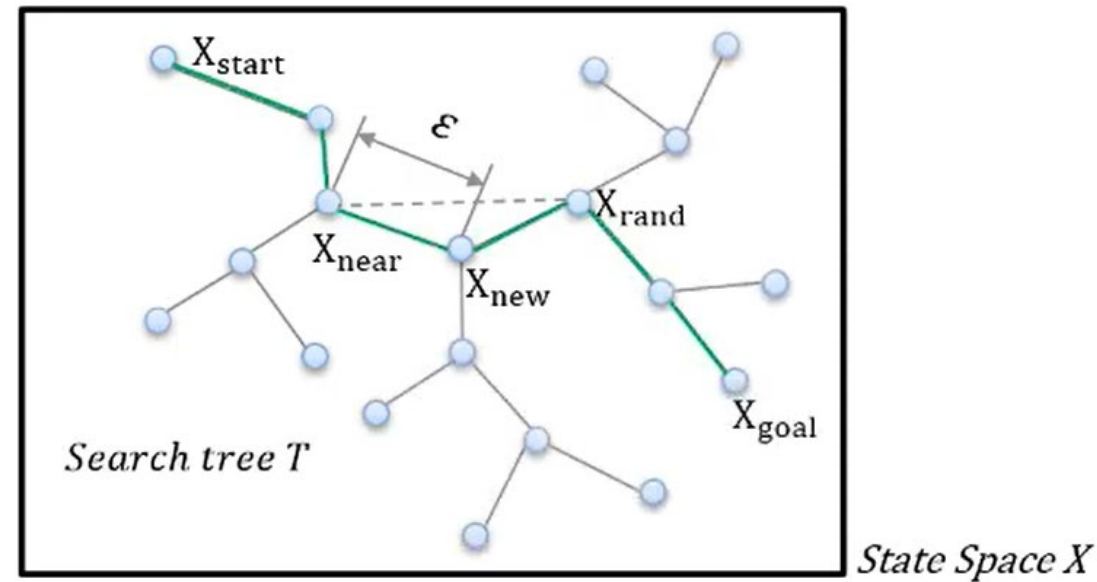
- Iterate** – Repeat steps 3–7 until the goal is reached.



METHODS & IMPLEMENTATIONS

Workings of RRT

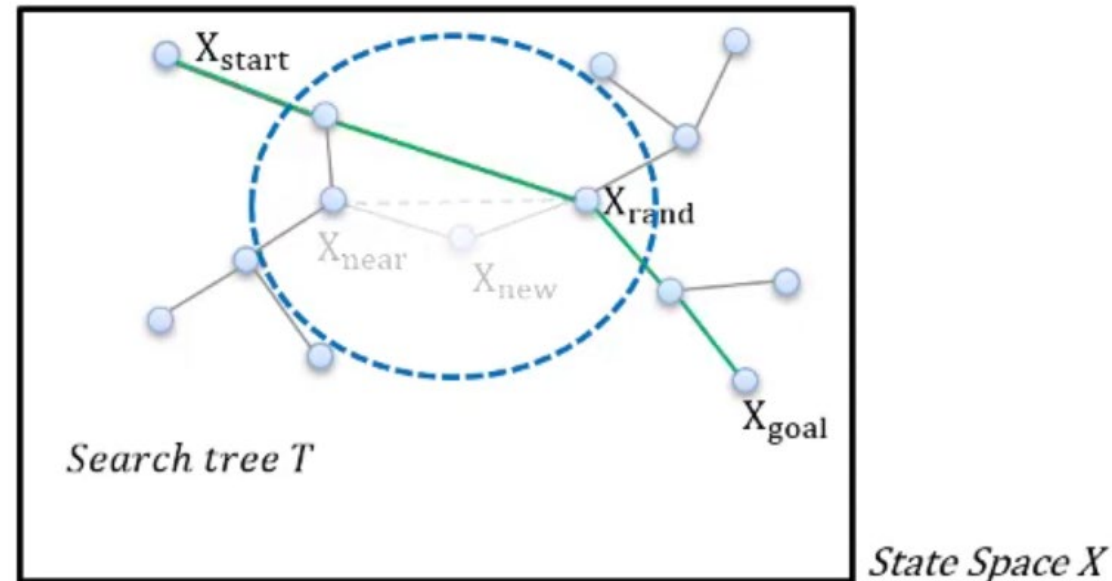
Path Extraction – Trace back through parent nodes to extract the path.



METHODS & IMPLEMENTATIONS

I RRT*

- I It expands the tree in a similar way like RRT.
- I It rewires the tree by looking at nearby nodes within a certain radius and tries to find a shorter path from the new node back to the start.





EXPERIMENTAL SETUP

■ Programming Language:

- MATLAB R2025a: High-level programming language, widely used for numeric computing, data analysis, algorithm development, simulation and visualization.

■ Toolboxes:

- Lidar Toolbox: MATLAB's toolbox that provides algorithms, apps, and tools for working with LiDAR data.
- Navigation Toolbox: Another MATLAB's toolbox that provides algorithms and analysis tools for motion planning, SLAM, and inertial navigation.



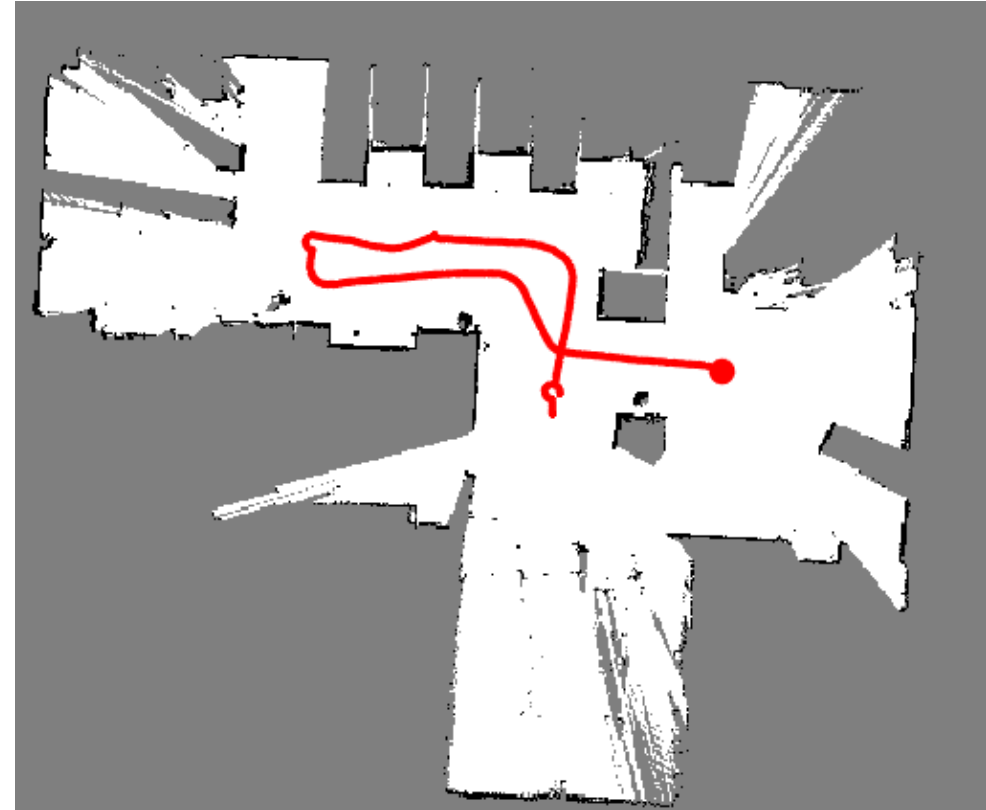
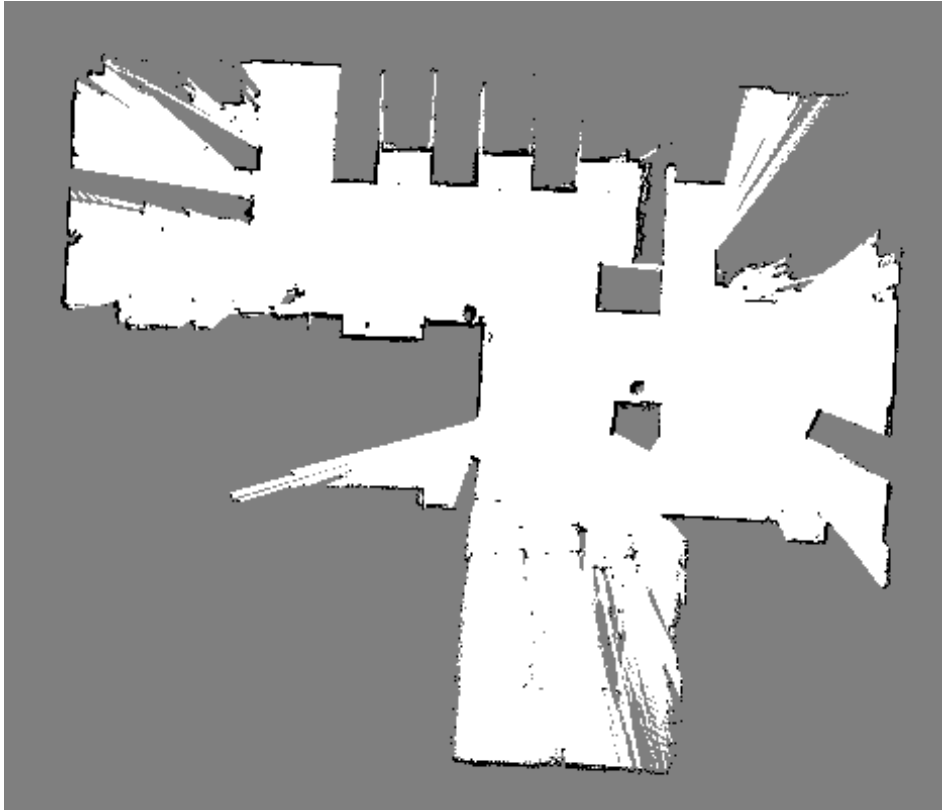
RESULTS

┃ Overview of Output Files From Map Generation Process:

- ┃ .mat File: Contains the occupancy grid map data.
- ┃ .yaml File: Stores image metadata for the occupancy grid map.
- ┃ .pgm File: Grayscale image format of the occupancy grid map.
- ┃ .png File: Grayscale image format of the occupancy grid map.

RESULTS

Generated Map of IOT Factory





RESULTS

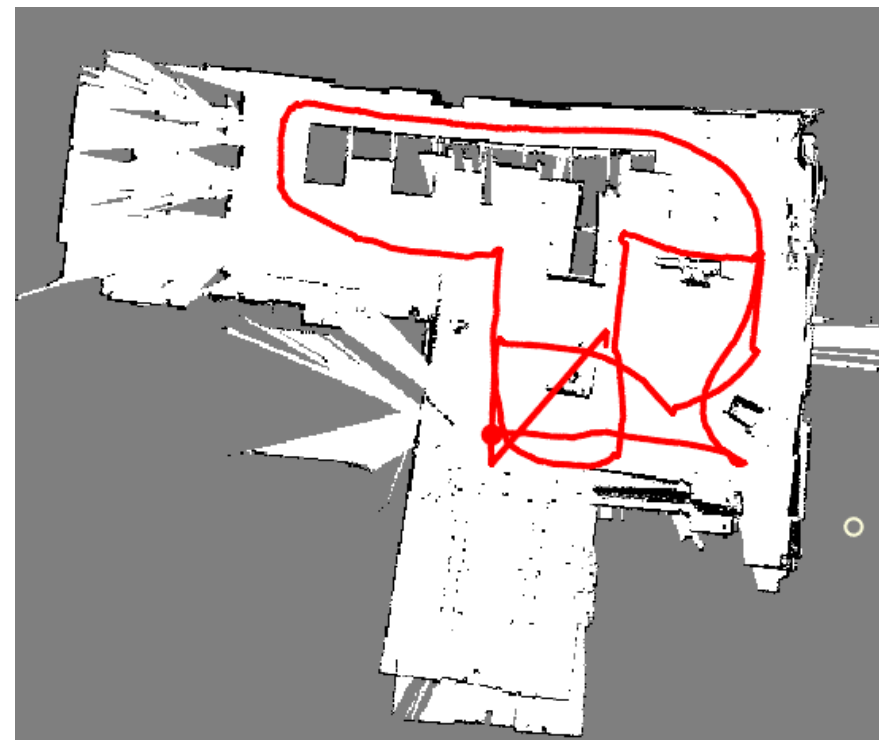
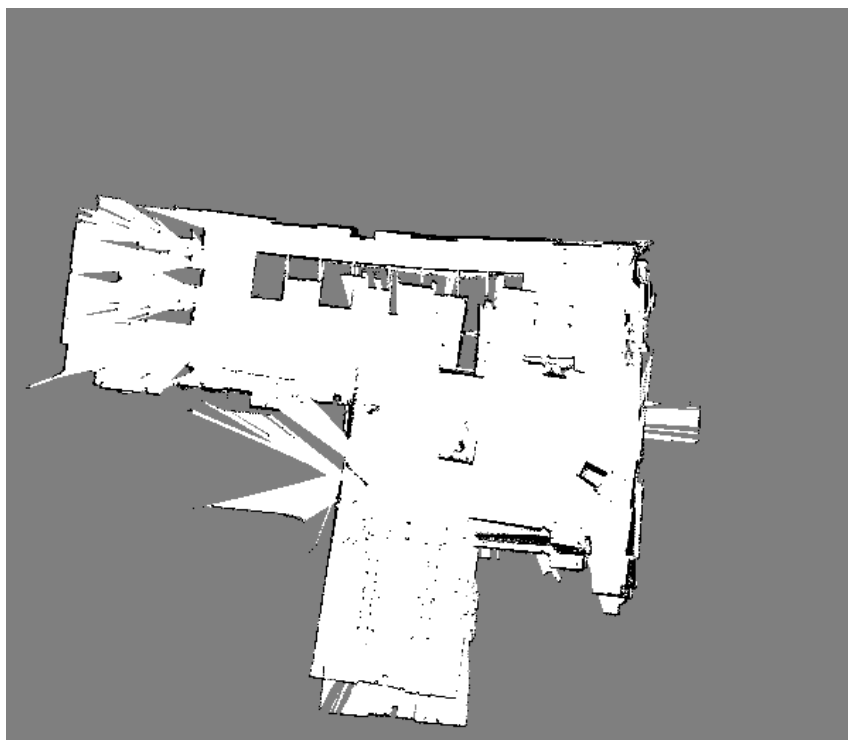
I Video of Map Generation of IOT Factory





RESULTS

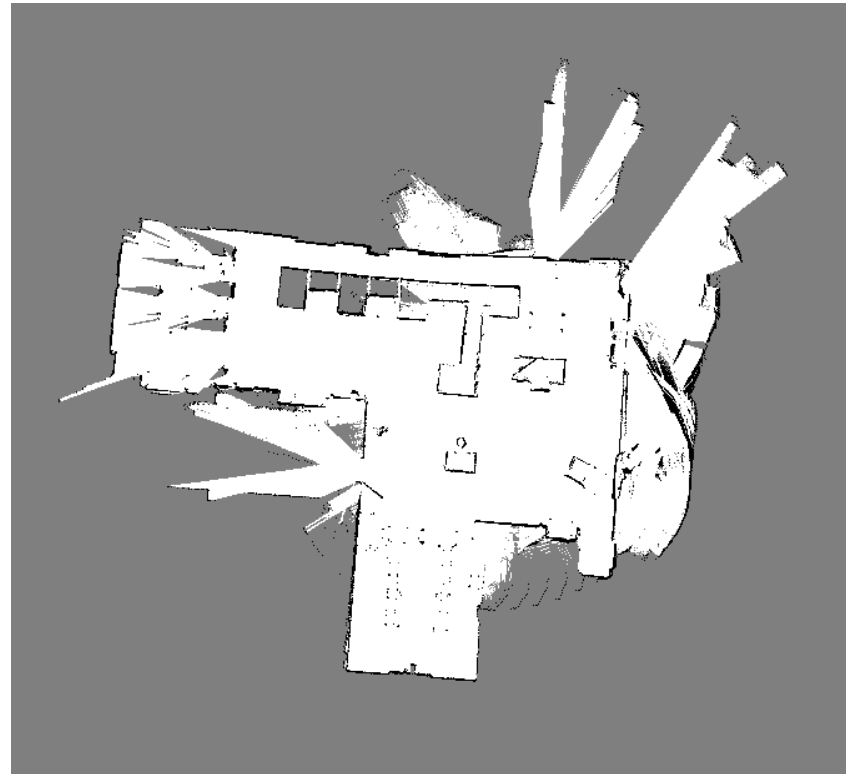
I Generated Map of IOT Factory with Different Data





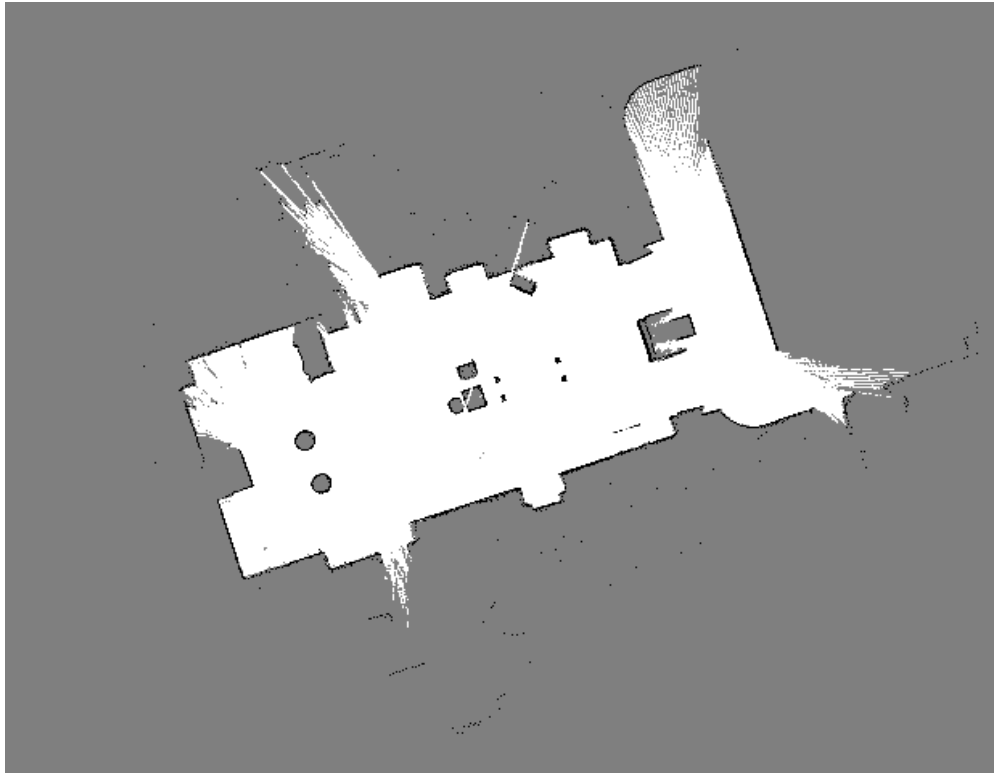
RESULTS

I Generated Map of IOT Factory with Different Data

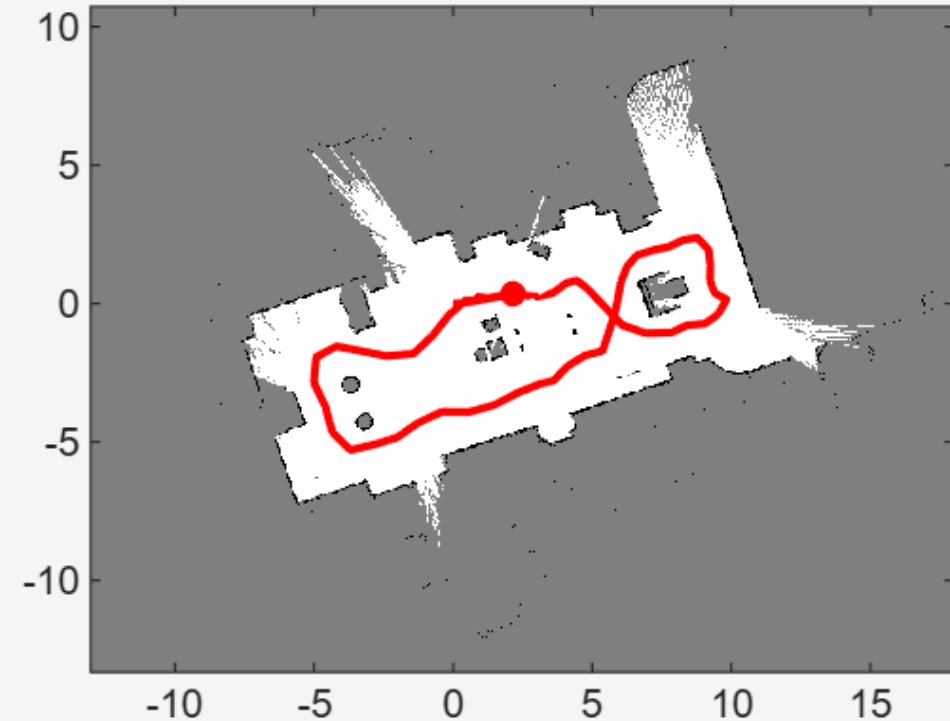


RESULTS

Generated Map of Warehouse



Warehouse Occupancy Map with Robot Trajectory





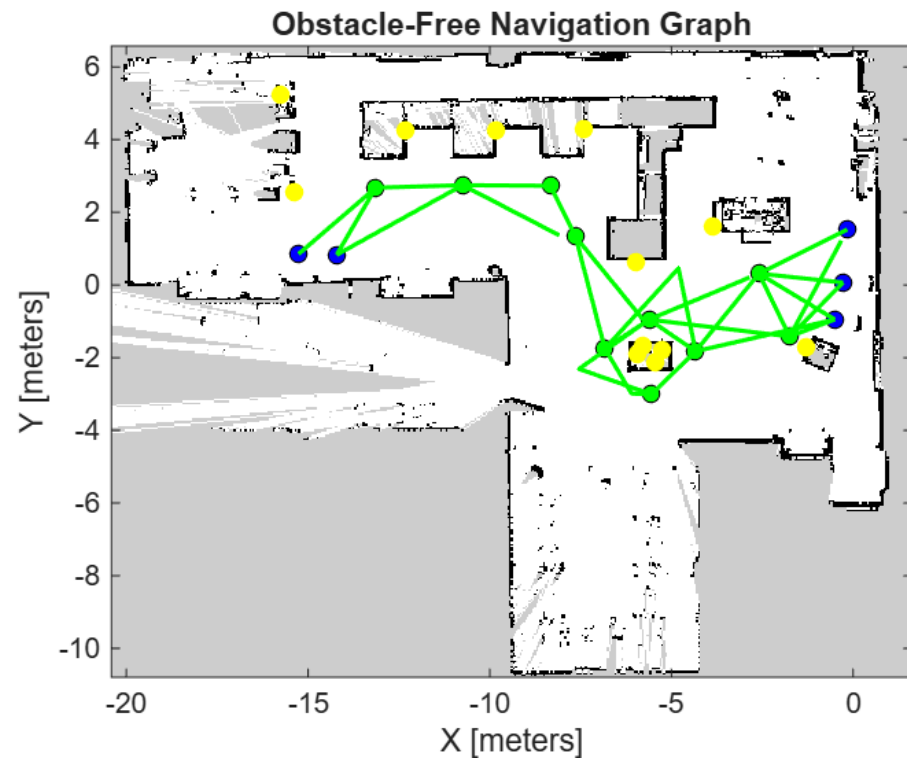
RESULTS

- Overview of Output Files From Path Network Generation Process:
 - navigation-paths.xml: contains navigation paths between nodes (docking+parking).
 - positions.lisp: A LISP file listing the coordinates of all nodes.
 - stations.lisp: A LISP file containing station coordinates and their closest associated nodes.

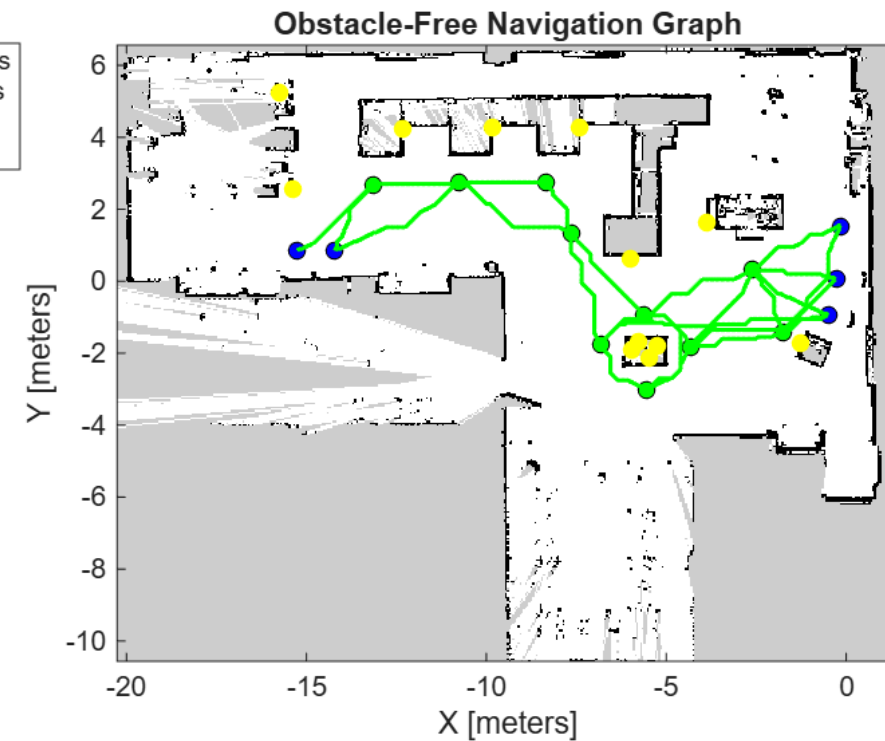
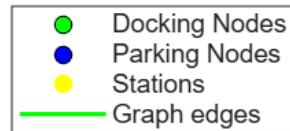


RESULTS

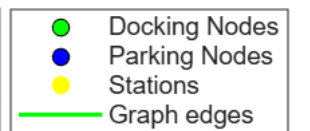
Path Network Generated by RRT* and A* on IOT Factory's Map



Generated by RRT*



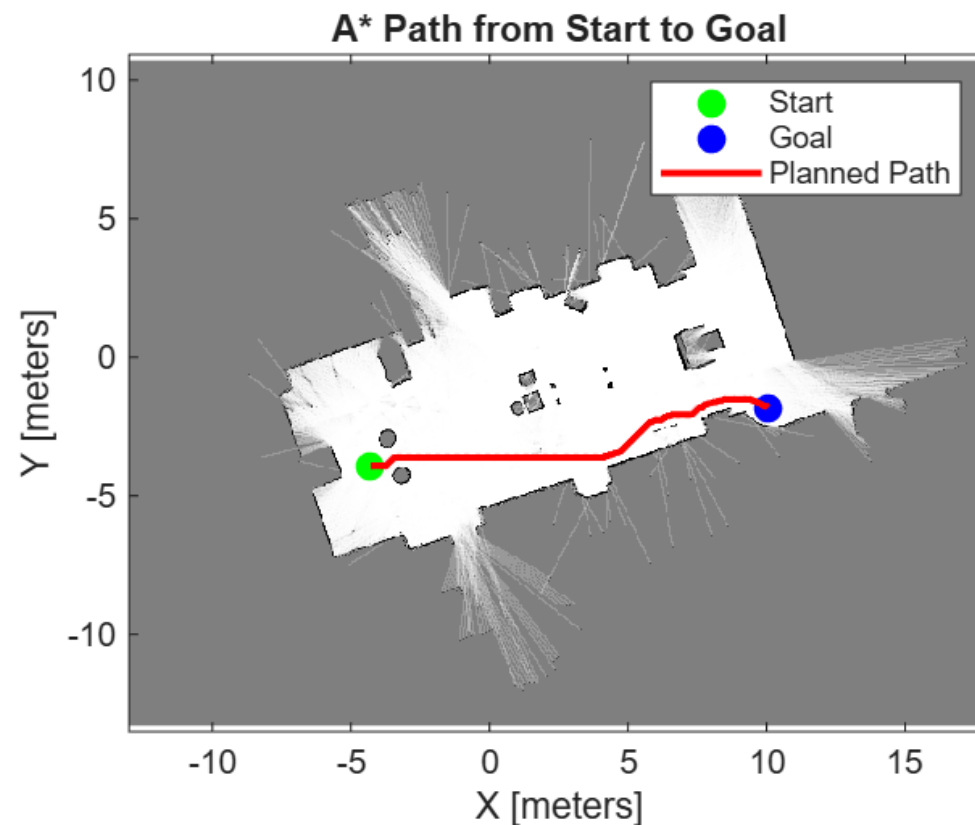
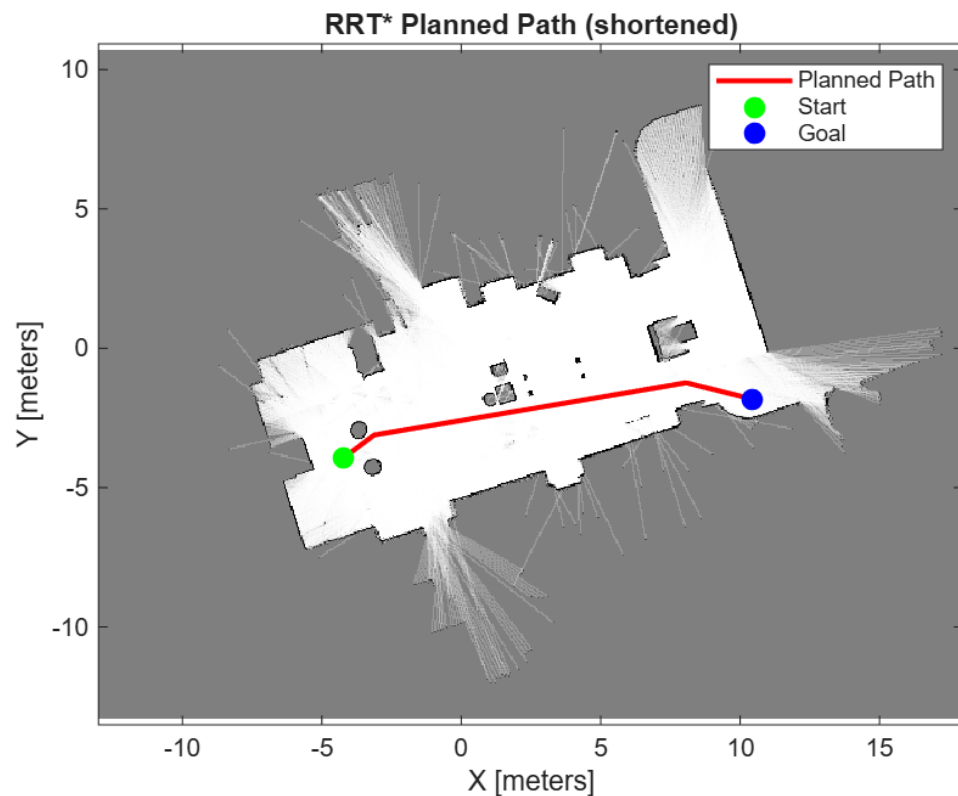
Generated by A*





RESULTS

Path Generated by RRT* and A* on Warehouse Map





CRITICAL REFLECTION

I Strengths of The Approach

- I The drift is mitigated through loop closure detection.
- I Pose graph optimization ensured globally consistent and robust map construction.
- I The A* algorithm produced optimal and shortest paths in the map.
- I The RRT* algorithm is also producing fairly optimal paths.



CRITICAL REFLECTION

■ Limitations

- Generated map of IOT factory is not better because:
 - It is only using LiDAR data
 - Sensor noise
 - LiDAR data collection not in a proper way
- Prebuilt map is using to generate optimized path network on the map.



CRITICAL REFLECTION

■ Challenges

- Getting an optimized path network on map requires extensive work:
 - Need to set world co-ordinate origin properly for the map
 - So that, all nodes, stations, and docking points must align properly on the map
 - Sometimes, the generated maps are tilted or oriented, then transformation of them is required.
- Converting the result of path network generation into Festo Software compatible format is difficult.
 - Performed custom mapping to get navigation-paths.xml, stations.lisp, and positions.lisp



CRITICAL REFLECTION

■ Future Work

- Integrating odometry and IMU data to generate better occupancy map.
- Using more sophisticated SLAM algorithms like Cartographer, LOAM, etc.
- Working with different path planning algorithms.
- Testing the generated path networks on Robotino's Festo Software.



